

Automated Library Entry & Occupancy Monitoring System

V1: RFID-based Entry/Exit, Digital Records, Live Occupancy & Analytics

Core idea

Replace the manual library entry register with a connected system that identifies students through RFID, stores entry/exit sessions digitally, shows live occupancy on a webpage, and converts historical records into useful library analytics.

V1 STACK ESP32 RC522 RFID Python SQLite HTML/CSS/JS Pandas / SQL Wi-Fi

Prepared as a build guide and portfolio project blueprint

1. Project Overview

The Automated Library Entry & Occupancy Monitoring System is a V1 IoT/embedded project for automating a common library workflow: identifying students at entry, recording entry and exit events, maintaining searchable records, showing current occupancy, and generating usage analytics. The system is intentionally designed as an end-to-end engineering project that connects hardware, networking, backend software, a database, and a browser-based interface.

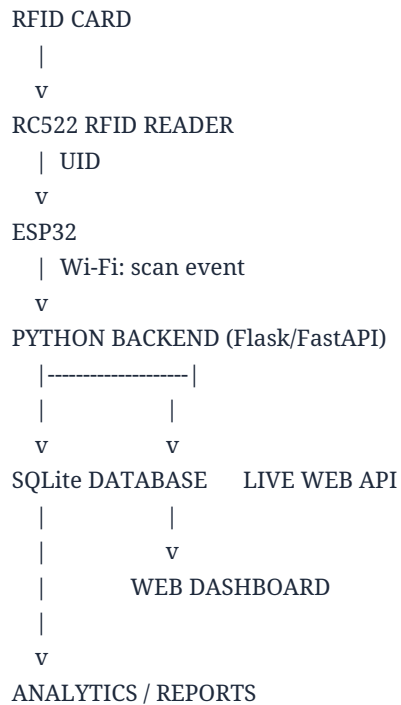
Area	V1 Goal
Problem	Manual register entry is slow, difficult to search later, and produces fragmented historical records.
Solution	Use RFID + ESP32 to capture identity and scan time, then use Python + SQLite to store and process sessions.
Live view	A webpage shows students currently inside, estimated available seats, and recent activity.
Analytics	Historical records are transformed into daily/monthly visits, peak hours, repeat usage, and duration metrics.
Portfolio value	Demonstrates embedded systems, automation logic, IoT communication, databases, backend development, web UI, and analytics.

2. V1 Scope - Four Modules

Module	Name	Purpose
Module A	RFID Identification & ESP32 Event Capture	Reads a student card/tag, retrieves its UID, debounces repeated scans, timestamps the event, and sends a scan to the backend over Wi-Fi.
Module B	Digital Library Database	Stores student profiles, RFID mappings, entry/exit sessions, raw events, and computed visit data in a structured database.
Module C	Live Web Dashboard	Shows current occupancy, estimated available seats, today's entries/exits, current library status, and recent scan activity.
Module D	Library Analytics	Uses historical records to generate visit counts, peak hours, average duration, busiest days, repeat usage, and report-ready summaries.

3. System Architecture

The V1 design separates responsibilities cleanly: the ESP32 handles the physical scan and network communication; the Python backend handles validation, entry/exit state, database writes, and APIs; the browser handles presentation; analytics uses the same database as its source of truth.



4. Component Responsibilities

RFID card/tag: Carries a unique identifier (RFID UID). Personal details should not be treated as the primary data payload on the card. The UID is linked to the student profile in the database.

RC522: Reads the RFID UID and passes it to the ESP32 through SPI communication.

ESP32: Controls the RFID reader, applies scan debounce/cooldown logic, connects to Wi-Fi, and sends scan events to the backend.

Python backend: Receives the scan, finds the student by RFID UID, decides entry/exit state, creates or closes a session, updates occupancy, and exposes data to the webpage.

SQLite: Stores structured records locally in a portable database file. It is a practical V1 choice and can later be replaced or upgraded to another database engine.

Web dashboard: Provides the human-facing interface for live occupancy and recent activity.

Analytics layer: Uses SQL and/or Pandas to calculate historical metrics and prepare charts/tables.

5. V1 Hardware Requirements

Component	Qty.	Role
ESP32 development board	1	Main microcontroller with Wi-Fi connectivity.
RC522 RFID reader module	1	Reads RFID UID from cards/tags.
RFID cards/tags	At least 2	One or more test cards for student identities.
Breadboard + jumper wires	1 set	Prototype wiring.
USB cable + 5 V supply	1	Power/programming for the ESP32 board.
Computer/server	1	Runs the Python backend, SQLite database, and local dashboard during V1.

Optional V2 hardware - seat sensors, DHT11/DHT22, light sensor, air-quality sensor, buzzer, servo-based gate/barrier, display, or additional RFID readers. These are deliberately outside the V1 scope.

6. Module A - RFID Identification & ESP32

6.1 Purpose

Module A is the hardware edge of the system. A student presents an RFID card at the reader. The RC522 reads the card UID, and the ESP32 packages the scan with a timestamp and device information before sending it to the Python backend over Wi-Fi.

6.2 Card model

The preferred design is to store only the RFID UID as the machine identifier. The printed card can visibly contain the student name, roll number, branch, and year, while the backend database holds the authoritative profile. This keeps the physical card simple and makes profile changes possible without rewriting card data.

Printed card

+-----+

| Name: Student Name |

| Roll No: AR26XXXX |

| Branch: Automation & |

| Robotics |

| Year: 1 |

+-----+

Machine identifier: RFID UID

6.3 Scan workflow

1. Card is presented
2. RC522 reads UID
3. ESP32 verifies that the scan is not a duplicate within the cooldown window
4. ESP32 sends UID + timestamp + device ID to Python backend
5. Backend validates UID against STUDENTS
6. Backend determines current state and records ENTRY or EXIT
7. Dashboard reflects the updated occupancy

6.4 Entry/exit logic for one reader

A practical V1 approach is state-based toggling. If the student has no open session, the next valid scan becomes ENTRY. If the student already has an open session, the next valid scan becomes EXIT. This avoids needing two physical readers for the first prototype.

Student status in DB	Next valid scan
OUT / no open session	-> ENTRY
IN / open session	-> EXIT
Unknown RFID	-> REJECT / LOG ERROR
Repeated scan < cooldown	-> IGNORE

Important edge case: one-reader state toggling assumes students do not scan accidentally or in the wrong order. The backend should therefore log raw events, reject unknown IDs, apply a short scan cooldown, and provide an admin correction method in later versions.

7. Module B - Database & Digital Records

7.1 Why a database instead of Excel?

Excel-like tables are useful for viewing and exporting, but a database is better as the system of record. A database gives structured relationships, safer updates, searchable history, and a clean path to APIs and analytics. V1 can use SQLite, with CSV export available for users who want spreadsheet-style records.

7.2 Recommended schema

Table	Important fields	Purpose
students	student_id, rfid_uid, name, roll_no, branch, year, active	Master student profile and RFID mapping.
library_sessions	session_id, student_id, entry_time, exit_time, duration_seconds, status	One visit/session from entry to exit.
event_log	event_id, student_id, rfid_uid, event_type, timestamp, device_id, accepted, reason	Immutable-style raw scan history for auditing and debugging.
settings	key, value	Library capacity and system

Table	Important fields	Purpose
		configuration.

7.3 Example session record

session_id: 1021
student_id: 1
entry_time: 2026-10-02 10:14:03
exit_time: 2026-10-02 11:52:17
duration_seconds: 5894
status: CLOSED

7.4 Search and export

The system should support filtering by student, roll number, branch, date range, and event type. A CSV export can reproduce an Excel-style register when required by the librarian or project evaluator.

8. Module C - Live Web Dashboard

8.1 Dashboard purpose

The dashboard converts database state into an easy-to-read live view. It should be designed for a librarian or visitor who wants to answer: how many people are inside, how many seats are estimated to be free, what is happening now, and how busy is the library today?

LIBRARY LIVE STATUS				
Students Inside	37			
Est. Empty Seats	23			
Capacity	60			
Entries Today	126			
Exits Today	89			
RECENT ACTIVITY				
AR001	Student A	10:32:18	ENTRY	
AR014	Student B	10:34:02	EXIT	

8.2 Occupancy calculation

For V1, occupancy is the number of students with an open library session. If the configured library capacity is 60 and 37 students are currently inside, the dashboard can show an estimated 23 available seats. This is an estimate unless actual seat sensors are added later.

Estimated available seats = Library capacity - Current students inside

8.3 Live update mechanism

A simple V1 implementation can poll a backend API every few seconds. Later, the same dashboard can be upgraded to WebSockets or Server-Sent Events for more immediate updates without changing the core data model.

9. Module D - Analytics

The analytics module turns raw sessions into information that helps understand library usage patterns. Analytics should be derived from recorded sessions rather than manually maintained counters.

Metric	What it tells you
Daily visits	Number of completed and/or started visits by date.
Unique visitors	Distinct students using the library during a selected period.
Peak hours	Hour blocks with the highest number of students/entries.
Average visit duration	Mean duration of completed sessions.
Busiest days	Days with the highest visit volume.
Repeat usage	Students with multiple visits during a period.
Occupancy trend	Occupancy measured at regular time intervals or reconstructed from sessions.

Suggested implementation: use SQL for straightforward aggregations and Pandas for more flexible data cleaning, time grouping, and export/report generation. Charts can be rendered in the browser using Chart.js or another simple visualization library.

10. Backend & Data Flow

The backend is the central coordinator. It should expose small, clear endpoints instead of allowing the ESP32 or browser to manipulate the database directly.

Endpoint	Purpose
POST /api/scan	ESP32 sends RFID UID, device ID, and scan timestamp.
GET /api/status	Returns current occupancy, capacity, estimated seats, and library status.
GET /api/recent-events	Returns recent accepted/rejected events for the dashboard.
GET /api/sessions	Returns filtered session history.

Endpoint	Purpose
GET /api/analytics	Returns precomputed or on-demand metrics for the analytics page.
GET /api/students/:id	Returns student profile and visit summary when needed.

10.1 Event payload example

```
{
  "rfid_uid": "A37F219C",
  "device_id": "LIB-ESP32-01",
  "scanned_at": "2026-10-02T10:32:18+05:30"
}
```

11. Suggested Software Structure

```
library-monitor/
|-- esp32/
|   |-- rfid_client.ino
|   `-- config.h
|
|-- backend/
|   |-- app.py
|   |-- database.py
|   |-- models.py
|   |-- services.py
|   `-- requirements.txt
|
|-- web/
|   |-- dashboard.html
|   |-- analytics.html
|   |-- css/
|   `-- js/
|
|-- data/
|   `-- library.db
|
`-- README.md
```

The exact folder structure can change during implementation. The important principle is to keep embedded code, backend code, frontend code, and database files separated so the project remains easy to maintain and present on GitHub.

12. Reliability, Privacy & Edge Cases

Case	Recommended handling
Unknown card	Reject the scan and store a reason such as UNKNOWN_RFID in event_log.
Duplicate tap	Ignore scans within a configurable cooldown window, for example a few seconds.
Wi-Fi unavailable	Queue a small number of unsent events in ESP32 memory or local storage and retry when connectivity returns. V1 can initially log the failure clearly.
Power restart	Database remains the source of truth. The ESP32 should reconnect and resume scanning without corrupting existing sessions.
Open session	A student who is marked IN remains counted in occupancy until an accepted EXIT scan closes the session.
Incorrect state	Provide a future admin correction feature rather than silently changing historical data.
Personal data	Store only the student information required for the project. Protect the database and do not expose unnecessary details on the public dashboard.

13. V1 Development Plan

Phase	Focus	Definition of done
Phase 1	ESP32 + RC522	Wire reader, detect cards, print UID, test multiple cards.
Phase 2	Python API	Create endpoint for scan events and test with a browser/Postman or Python client.
Phase 3	SQLite	Create tables, register students, write accepted/rejected events, open/close sessions.
Phase 4	ESP32 to backend	Send real scan events over Wi-Fi and confirm database updates.
Phase 5	Live dashboard	Create status cards, occupancy, estimated seats, counters, and recent activity.
Phase 6	Analytics	Add historical tables/charts, peak hours, visit counts, duration, and

Phase	Focus	Definition of done
		filters.
Phase 7	Testing + documentation	Test repeated taps, unknown cards, restarts, Wi-Fi loss, and incorrect states. Document setup and architecture.

14. V1 Acceptance Tests

Test	Expected result
Valid entry	Known RFID card with no open session creates an ENTRY event and opens a session.
Valid exit	Same student with open session creates an EXIT event and closes the session.
Occupancy	Current students inside equals the number of open sessions.
Duplicate prevention	Repeated taps inside the cooldown window do not create duplicate sessions/events.
Unknown RFID	Unregistered card is rejected and logged without changing occupancy.
Dashboard	Live status and recent activity change after a valid scan.
Analytics	Historical scans produce consistent daily, hourly, and duration statistics.
Restart	After restarting the ESP32 or backend, existing database records remain intact and the system resumes normal operation.

15. Future Expansion - V2 and Beyond

V1 should remain focused and reliable. Once A-D work end-to-end, the architecture can be extended without changing the fundamental project idea.

Actual seat occupancy: Add seat-level IR/ToF/pressure/other suitable sensors so the dashboard can show real empty seats rather than an estimate.

Environment monitoring: Temperature, humidity, light, air-quality, and noise sensors can provide a richer library-condition view.

Automated gate/barrier: Use a servo or other appropriate actuator to open a miniature/controlled entry mechanism after successful identification.

Two-reader entry/exit: Separate physical readers remove ambiguity between entry and exit and make the state machine more robust.

Cloud deployment: Move the backend/database to a hosted environment if remote access is required.

Authentication: Add librarian/admin login and role-based access to student records and analytics.

16. Portfolio Presentation

For a portfolio, present the project as a real-world automation problem and show the complete system rather than only the final webpage. The strongest evidence is a short demo: scan a card, show the backend event, show the database record, then show the live occupancy change and an analytics chart.

Portfolio item	Content
Project title	Automated Library Entry & Occupancy Monitoring System
One-line pitch	An ESP32 + RFID + Python system that automates library entry/exit records and turns live and historical usage into a monitoring dashboard and analytics.
Engineering themes	Embedded systems, IoT, automation logic, database design, API development, web dashboards, data analytics.
Demo sequence	RFID tap -> ESP32 -> backend -> database -> occupancy change -> analytics/history.
GitHub evidence	README, system architecture, circuit diagram, database schema, source code, screenshots, sample data, testing notes, and demo video/GIF.

17. V1 Definition of Done

V1 is complete when a known RFID card can be scanned at the RC522, the ESP32 can send the event over Wi-Fi, the Python backend can identify the student and open/close a library session, the SQLite database can store and retrieve the record, the live dashboard can show the updated occupancy and recent activity, and the analytics page can summarize historical usage.

End-to-end success condition: CARD TAP -> IDENTIFY -> RECORD -> UPDATE OCCUPANCY -> DISPLAY -> ANALYZE

Implementation notes: Use test student data during development; keep real personal data out of public repositories. Keep library capacity configurable, retain raw scan events for debugging, treat V1 seat availability as an estimate, and demonstrate the system first as a desk prototype.